Le layout avec CSS Grid

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Qu'est-ce qu'un CSS Grid layout ? **4**

A. Qu'est-ce qu'un CSS Grid layout ? 4

B. Exercice : Quiz 6

III - Grid et mises en page complexes **8**

A. Grid et mises en page complexes 8

B. Exercice : Quiz 12

IV - Essentiel **14**

V - Auto-évaluation **15**

A. Exercice 15

B. Test 16

Solutions des exercices **18**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 heure

Prérequis : bases de HTML / CSS - notions flex

Environnement de travail : IDE - LiveServer sur VS Code

Contexte

Lorsqu'il s'agit de créer des sites web modernes et esthétiquement attrayants, la combinaison du HTML et des grilles CSS est essentielle. Le HTML, acronyme de *HyperText Markup Language*, est le langage de balisage standard utilisé pour structurer et présenter le contenu d'une page web. Il offre une base solide en fournissant une structure logique à nos documents web, définissant les éléments et leur hiérarchie. D'un autre côté, les grilles CSS, ou *Cascading Style Sheets*, sont un outil puissant pour la mise en page des sites web. Les grilles CSS permettent aux concepteurs de créer des mises en page flexibles et *responsives*, où les éléments de la page peuvent être organisés en colonnes et en lignes. Cela permet une grande souplesse pour l'agencement des éléments et facilite l'adaptation de la disposition en fonction de différentes tailles d'écran.

Tous droits réservés à STUDI - Reproduction interdite

II Qu'est-ce qu'un CSS Grid layout ?

A. Qu'est-ce qu'un CSS Grid layout ?

Az Définition

Un Grid layout en CSS est une méthode de mise en page utilisée pour créer des designs harmonieux et structurés pour les sites web. Il s'agit d'une grille invisible qui divise une page en colonnes et en rangées régulières, ce qui permet aux éléments de la page d'être alignés avec précision. Les Grid layouts en CSS sont populaires, car ils permettent une plus grande cohérence dans la conception, une meilleure organisation de l'information et une expérience utilisateur plus agréable.

L'utilité d'un layout avec CSS Grid

Pourquoi utiliser un Grid layout en CSS ? Il y a plusieurs raisons. Tout d'abord, il permet d'**aligner** facilement les éléments de la page de manière cohérente, ce qui donne un aspect plus professionnel et structuré. En outre, il permet de créer des **mises en page réactives**, ce qui signifie que la page peut s'adapter à différentes tailles d'écran et à différents appareils : c'est essentiel pour une expérience utilisateur optimale. Enfin, les Grid layouts en CSS facilitent la **mise à jour et la maintenance** de la page, car ils permettent de modifier rapidement et facilement la disposition de la page sans avoir à modifier manuellement chaque élément.

Maintenant que nous avons compris ce qu'est un Grid layout en CSS et pourquoi il est utile, nous allons passer en revue les concepts de base pour comprendre comment les utiliser efficacement avec quelques exemples.

Exemple

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Exemple 1 : Layout grid à deux colonnes</title>
    <!-- CSS -->
    <style>
      /* Définition du conteneur de grille */
      .container {
        display: grid;
        grid-template-columns: repeat(2, 1fr); /* Deux colonnes de largeur égale */
        gap: 20px; /* Espacement de 20 pixels entre les éléments */
      }

      /* Style des éléments de la grille */
      .item {
        background-color: #e5e5e5; /* Couleur de fond des éléments */
        padding: 20px; /* Espacement intérieur de 20 pixels */
      }
    </style>
  </head>
  <body>
    <!-- Conteneur de grille -->
    <div class="container">
      <!-- Élément de la colonne 1 -->
```


Tous droits réservés à STUDI - Reproduction interdite

```
      <div class="item">Contenu de la colonne 1</div>

      <!-- Élément de la colonne 2 -->
      <div class="item">Contenu de la colonne 2</div>
    </div>
  </body>
</html>
```

Ci-dessus, vous pouvez voir un exemple de code HTML pour créer un Grid layout à 2 colonnes. Tout d'abord, dans l'en-tête, le CSS est inclus dans le code HTML. Cependant, il est généralement recommandé d'utiliser un fichier CSS externe pour séparer le style de la présentation. Dans le corps du document, nous avons un conteneur avec la classe `container` et 2 éléments enfants avec la classe `item`.

Dans notre feuille de style CSS, nous avons créé une grille en déclarant la propriété `display` comme « ***grid*** » pour notre conteneur. Nous avons également défini le nombre de colonnes à 2 en utilisant la propriété `grid-template-columns` avec la fonction `repeat(2, 1fr)`. La fonction CSS `repeat()` offre une solution pratique pour répéter une ou plusieurs valeurs un certain nombre de fois. Elle se compose de 2 arguments : le premier indique combien de fois la valeur doit être répétée, tandis que le second est la valeur à répéter. La fonction `1fr` signifie que chaque colonne occupe la même quantité d'espace disponible. Nous avons également défini un espacement de 20 pixels entre les éléments enfants en utilisant la propriété `grid-gap`. Enfin, nous avons appliqué un fond de couleur gris clair à chaque élément enfant et ajouté un rembourrage de 20 pixels pour les espacer du bord de la colonne. Ce code est facilement personnalisable en modifiant le nombre de colonnes, l'espacement et le style visuel des éléments enfants en fonction de vos besoins.

Méthode

Dans l'exemple ci-dessus, nous avons vu comment créer une grille de base en CSS. Pour cela, nous avons utilisé la propriété `display` avec la valeur « *grid* » appliquée à notre conteneur. Pour ajouter des colonnes et des lignes à la grille, nous utilisons la propriété `grid-template-columns` pour les colonnes et `grid-template-rows` pour les lignes. Nous pouvons spécifier la taille de chaque colonne ou ligne en utilisant des unités de mesure telles que les pixels, pour une taille fixe, ou les pourcentages, pour une taille relative à la largeur de la grille.

```
.container {
  display: grid; /* Déclare un conteneur de grille */
  grid-template-columns: 1fr 1fr; /* Définit deux colonnes de largeur égale */
  grid-template-rows: 50px 50px; /* Définit deux lignes de hauteur fixe de 50 pixels */
}
```

Dans cet exemple, nous avons créé une grille avec 2 colonnes de taille égale et 2 lignes de 50 pixels de hauteur chacune. Pour ajouter des espaces entre les colonnes et les lignes, nous allons utiliser la propriété `grid-gap`. Cette propriété nous permet de spécifier l'espacement entre les colonnes et les lignes en utilisant des unités de mesure telles que les pixels ou les em.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Voici un exemple de code pour ajouter un espacement de 20 pixels entre les colonnes et les lignes de notre grille :

```
.container {
  display: grid; /* Déclare un conteneur de grille */
  grid-template-columns: 1fr 1fr; /* Définit deux colonnes de largeur égale */
  grid-template-rows: 50px 50px; /* Définit deux lignes de hauteur fixe de 50 pixels */
  gap: 20px; /* Espacement de 20 pixels entre les éléments de la grille */
}
```

En vue de personnaliser davantage notre grille, nous pouvons intégrer des marges en appliquant la propriété `margin` à notre conteneur.

Exemple

Prenons l'exemple où nous souhaitons ajouter une marge de 20 pixels autour de notre grille. Dans ce cas, nous pouvons ajouter la règle suivante à notre feuille de style CSS :

```
.container {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 20px;
  margin: 20px;
}
```

Cette règle ajoute une marge de 20 pixels autour de notre conteneur de grille, ce qui lui donne un aspect plus aéré et permet aux éléments enfants de mieux se démarquer. De plus, cette règle est facilement modifiable pour ajuster la taille et l'emplacement de la marge en fonction de nos besoins de conception.

B. Exercice : Quiz

[solution n°1 p. 18]

Question 1

Qu'est-ce que CSS Grid ?

- Une technique pour créer des grilles de données dans Excel
- Un langage de programmation utilisé pour développer des applications web
- Un module CSS qui permet de créer des mises en page en utilisant des grilles

Question 2

Quels sont les 2 composants principaux de CSS Grid ?

- Lignes et colonnes
- Classes et identifiants

Tous droits réservés à STUDI - Reproduction interdite

Divs et spans

Question 3

Quelle propriété CSS permet de définir les lignes d'une grille en CSS Grid ?

`grid-template-rows`

`grid-layout`

`grid-container`

Question 4

Quelle est la façon la plus optimisé pour spécifier la taille d'une colonne ou d'une ligne dans CSS Grid ?

En utilisant des pourcentages

En utilisant des pixels

En utilisant des fractions (fr)

Question 5

Comment peut-on définir un espace entre les éléments d'une grille dans CSS ?

En utilisant la propriété `grid-gap`

En utilisant la propriété `margin`

En utilisant la propriété `padding`

Tous droits réservés à STUDI - Reproduction interdite

III Grid et mises en page complexes

A. Grid et mises en page complexes

Méthode

Avec les techniques avancées de Grid layout en CSS, il est possible de créer des mises en page de type « *magazine* ». Tout d'abord, il est important de définir les différents éléments de la mise en page tels que les zones de texte, les images et les blocs de contenu. Ensuite, il faut utiliser les propriétés de Grid layout telles que `grid-template-areas` et `grid-template-columns` pour créer une grille flexible et positionner ces éléments. En prenant l'exemple d'une mise en page de magazine avec une couverture, une table des matières, des articles et des publicités, nous pouvons créer une grille de base en utilisant `grid-template-columns` pour définir le nombre et la largeur des colonnes, et `grid-template-rows` pour définir la hauteur des lignes. Pour créer des zones de contenu nommées pour chaque élément de la mise en page, la propriété `grid-template-areas` peut être utilisée. Par exemple, nous pouvons définir une zone pour la couverture, une zone pour la table des matières, une zone pour chaque article et une zone pour les publicités.

Une fois que les zones de contenu sont définies, chaque élément peut être placé dans la grille à l'aide de la propriété `grid-area` en utilisant les noms des zones de contenu définies. Par exemple, pour la couverture, nous allons définir la zone « *cover* » qui couvrira toutes les colonnes et 2 lignes. Pour la table des matières, nous définirons la zone « *mat* » qui couvre les 3 premières colonnes et 3 lignes. Pour chaque article, nous allons définir une zone nommée « *article1* » et « *article2* » qui couvrira 3 colonnes et 2 lignes. Pour les publicités, nous choisirons 2 zones nommées « *pub1* » et « *pub2* » qui couvriront les 2 dernières colonnes et 1 ou 2 lignes. Avec ces techniques de Grid layout avancées, il est possible de créer des mises en page de type « *magazine* » complexes et professionnelles.

Exemple

Voici le code HTML pour une mise en page de type « *magazine* » :

```html
<!DOCTYPE html>
<html>

<head>
    <title>Mise en page de type "magazine"</title>
    <link rel="stylesheet" href="css/style.css">
</head>

<body>
    <div class="container">
        <div class="cover">Couverture</div>
        <div class="mat">Table des matières</div>
        <div class="article1">Article 1</div>
        <div class="article2">Article 2</div>
        <div class="pub1">Publicité 1</div>
        <div class="pub2">Publicité 2</div>
    </div>
</body>

</html>
```


Tous droits réservés à STUDI - Reproduction interdite

La page inclut une section « *head* » avec un titre et une référence à un fichier CSS, ainsi qu'une section « *body* » avec une grille contenant 6 éléments : la couverture, la table des matières, 2 articles et 2 publicités. Les noms de classe pour chaque élément correspondent aux noms de classe utilisés dans le fichier CSS pour appliquer les styles de mise en page.

Voici ici nos règles CSS :

```css
/* Classe container */
.container {
  display: grid; /* Déclare un conteneur de grille */
  grid-template-columns: repeat(12, 1fr); /* Définit 12 colonnes de largeur égale */
  grid-template-rows: 80px repeat(4, minmax(0, 1fr)); /* Définit les hauteurs des lignes */
  gap: 20px; /* Espacement de 20 pixels entre les éléments de la grille */
}

/* Classe couverture */
.cover {
  grid-column: 1 / -1; /* Couvre toutes les colonnes */
  grid-row: 1 / 3; /* Couvre deux lignes */
  background-color: #c9c9c9; /* Couleur de fond */
  text-align: center; /* Alignement du texte au centre */
  padding: 20px; /* Espacement intérieur */
  font-size: 24px; /* Taille de police */
  font-weight: bold; /* Gras */
  color: #fff; /* Couleur du texte */
}

/* Classe table des matières */
.mat {
  grid-column: 1 / 4; /* Occupe trois colonnes */
  grid-row: 3 / 6; /* Occupe trois lignes */
  background-color: #f2f2f2; /* Couleur de fond */
  padding: 20px; /* Espacement intérieur */
}

/* Classe des articles */
.article1 {
  grid-column: 4 / 7; /* Occupe trois colonnes */
  grid-row: 3 / 5; /* Occupe deux lignes */
  background-color: #c9c9c9; /* Couleur de fond */
  padding: 20px; /* Espacement intérieur */
  color: #fff; /* Couleur du texte */
}

.article2 {
  grid-column: 7 / 10; /* Occupe trois colonnes */
  grid-row: 3 / 5; /* Occupe deux lignes */
  background-color: #c9c9c9; /* Couleur de fond */
  padding: 20px; /* Espacement intérieur */
  color: #fff; /* Couleur du texte */
}

/* Classes des publicités */
.pub1 {
  grid-column: 10 / -1; /* Commence à la colonne 10 et s'étend jusqu'à la fin */
  grid-row: 3 / 4; /* Occupe une ligne */
  background-color: #f2f2f2; /* Couleur de fond */
  padding: 20px; /* Espacement intérieur */
}

.pub2 {
  grid-column: 10 / -1; /* Commence à la colonne 10 et s'étend jusqu'à la fin */
```

Tous droits réservés à STUDI - Reproduction interdite

```
  grid-row: 5 / 6; /* Occupe une ligne */
  background-color: #f2f2f2; /* Couleur de fond */
  padding: 20px; /* Espacement intérieur */
}
```

Ces règles CSS permettent de créer une mise en page de magazine grâce à une grille CSS et à la **classe `container`**. Cette classe définit une grille de 12 colonnes et 5 rangées, dont la première rangée est dédiée à la couverture tandis que les 4 autres rangées accueillent la table des matières, les 2 articles et les 2 publicités.

La couverture est gérée par la classe `cover`. Elle occupe toutes les colonnes de la grille (`grid-column: 1 / -1`) ainsi que les 2 premières rangées (`grid-row: 1 / 3`). Cette classe a un arrière-plan gris clair (`#c9c9c9`), du texte centré, une taille de police de 24 px et une couleur de texte blanche.

La table des matières est gérée par la classe `mat`. Elle occupe les 3 premières colonnes (`grid-column: 1 / 4`) ainsi que les 3 dernières rangées (`grid-row: 3 / 6`). Cette classe a un arrière-plan gris clair (`#f2f2f2`) et une marge intérieure de 20 px.

Les classes `article1` et `article2` sont utilisées pour les 2 articles. Elles occupent chacune 3 colonnes et 2 rangées. La première classe occupe les colonnes 4 à 6 (`grid-column: 4 / 7`) et les rangées 3 à 4 (`grid-row: 3 / 5`), tandis que la seconde occupe les colonnes 7 à 9 (`grid-column: 7 / 10`) et les rangées 3 à 4 (`grid-row: 3 / 5`). Ces classes ont toutes les deux un arrière-plan gris foncé (`#c9c9c9`), une marge intérieure de 20 px et une couleur de texte blanche.

Enfin, **les classes `pub1` et `pub2`** sont utilisées pour les 2 publicités. Elles occupent chacune les colonnes 10 à la fin de la grille (`grid-column: 10 / -1`). La première classe occupe la rangée 3 (`grid-row: 3 / 4`), tandis que la seconde occupe la rangée 5 (`grid-row: 5 / 6`). Ces classes ont toutes les deux un arrière-plan gris clair (`#f2f2f2`) et une marge intérieure de 20 px.

L'adaptation aux différents types d'écrans

Méthode

Lorsque vous utilisez CSS Grid pour créer une mise en page adaptative, vous pouvez facilement ajuster votre grille en fonction des différents types d'écrans. Pour ce faire, vous pouvez utiliser des ***medias queries***. Les medias queries sont des instructions en CSS qui permettent d'appliquer des règles spécifiques en fonction des caractéristiques de l'écran, comme sa largeur, sa hauteur ou son orientation. Prenons un exemple concret : supposons que vous ayez une grille avec 3 colonnes pour les écrans larges et 1 seule colonne pour les écrans étroits. Vous pouvez utiliser les medias queries pour définir ces configurations spécifiques.

Exemple

```
/* Grille pour les écrans larges */
@media (min-width: 1024px) {
  .grid-container {
    grid-template-columns: 1fr 1fr 1fr; /* Définit trois colonnes de largeur égale */
  }
}

/* Grille pour les écrans étroits */
@media (max-width: 767px) {
  .grid-container {
```


Tous droits réservés à STUDI - Reproduction interdite

```
    grid-template-columns: 1fr; /* Définit une colonne de largeur égale */
  }
}
```

Dans cet exemple, nous utilisons la classe `.grid-container` pour créer une grille. À l'intérieur des medias queries, nous pouvons définir des styles spécifiques pour la grille en fonction de la taille de l'écran. Cette approche nous permet d'adapter facilement la grille aux différents types d'écrans, ce qui offre une meilleure expérience utilisateur sur différents appareils. Il est important de tester la grille sur différents types d'écrans pour s'assurer qu'elle s'ajuste correctement. Vous pouvez utiliser les outils de développement de votre navigateur pour visualiser votre mise en page sur différents appareils et apporter les ajustements nécessaires afin d'obtenir le résultat souhaité. En faisant cela, vous vous assurez que votre grille est réactive et convient à une variété de tailles d'écrans.

Les unités de mesure adaptatives, telles que `fr` (fraction) et `minmax()`, sont utiles pour créer des grilles flexibles et adaptatives. Prenons un exemple concret : vous pouvez définir une grille où chaque colonne a la même largeur en utilisant l'unité `fr`. Cela signifie que l'espace disponible dans le conteneur sera réparti de manière égale entre les colonnes.

Exemple

```
.grid-container {
  display: grid; /* Déclare un conteneur de grille */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); /* Définit les colonnes de la grille */
}
```

Dans cet exemple, nous utilisons la classe `.grid-container` pour créer notre grille. La propriété `grid-template-columns` est configurée avec `repeat(auto-fit, minmax(200px, 1fr))`. Ce paramètre permet aux colonnes de se répéter automatiquement et d'avoir une largeur minimale de 200 pixels, tout en utilisant l'unité `fr` pour une largeur flexible et remplir l'espace disponible. Cette approche assure que les colonnes de la grille s'adaptent de manière automatique en fonction de la taille de l'écran. Ainsi, votre mise en page s'ajuste de manière fluide et flexible, peu importe le type d'écran utilisé par l'utilisateur. Cela signifie que votre contenu sera réparti de manière équilibrée et harmonieuse, optimisant l'utilisation de l'espace disponible sur l'écran, qu'il s'agisse d'un grand écran ou d'un écran plus petit. Cela contribue à offrir une expérience utilisateur agréable et adaptée à différentes tailles d'écrans.

Méthode

```
<!DOCTYPE html>
<html>

<head>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
    <div class="grid-container">
        <div class="grid-item">Item 1</div>
        <div class="grid-item">Item 2</div>
        <div class="grid-item">Item 3</div>
        <div class="grid-item">Item 4</div>
        <div class="grid-item">Item 5</div>
        <div class="grid-item">Item 6</div>
    </div>
```

Tous droits réservés à STUDI - Reproduction interdite

```
</body>
</html>
```

```
.grid-container {
  display: grid; /* Déclare un conteneur de grille */
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); /* Définit les colonnes de la grille */
  grid-gap: 10px; /* Espacement entre les éléments de la grille */
}

.grid-item {
  background-color: #ccc; /* Couleur de fond des éléments de la grille */
  padding: 20px; /* Espacement intérieur des éléments de la grille */
  text-align: center; /* Alignement du texte au centre des éléments de la grille */
}

@media screen and (max-width: 767px) {
  /* Règles pour les écrans étroits avec une largeur maximale de 767 pixels */
  .grid-container {
    grid-template-columns: 1fr; /* Définit une seule colonne pour les écrans étroits */
  }
  .grid-item {
    background-color: red; /* Couleur de fond des éléments de la grille pour les écrans étroits */
  }
}
```

Dans cet exemple, nous utilisons une classe CSS appelée `.grid-container` pour créer un conteneur qui servira de grille. À l'intérieur de ce conteneur, nous avons des éléments avec la classe `.grid-item` qui représentent les éléments individuels de la grille. Pour définir la grille, nous utilisons la propriété CSS `grid-template-columns` avec la valeur `repeat(auto-fit, minmax(200px, 1fr))`.

Cette configuration permet aux colonnes de se répéter automatiquement et d'avoir une largeur minimale de 200 pixels, tout en se redimensionnant de manière flexible pour remplir l'espace disponible dans le conteneur. Nous ajoutons également une propriété `grid-gap` avec une valeur de 10 pixels pour créer un espacement entre les éléments de la grille.

De plus, nous utilisons un media query pour adapter la grille lorsque la largeur de l'écran est inférieure à 768 pixels. À ce moment-là, nous modifions la propriété `grid-template-columns` en `1fr`, ce qui signifie que la grille aura une seule colonne. En utilisant cet exemple, la grille s'ajuste automatiquement en fonction de la taille de l'écran, offrant ainsi une expérience utilisateur optimale sur différents appareils. Vous pouvez ajouter davantage d'éléments à la grille ou personnaliser les styles en fonction de vos besoins spécifiques.

B. Exercice : Quiz

[solution n°2 p. 19]

Question 1

Comment définir une grille à 3 colonnes pour les écrans de largeur supérieure à 1 200 pixels en utilisant les media queries ?

```
@media (min-width: 1200px) { grid-template-columns: 1fr 1fr 1fr; }
```

```
@media (max-width: 1200px) { grid-template-columns: 1fr 1fr 1fr; }
```


Tous droits réservés à STUDI - Reproduction interdite

```
@media (min-width: 1200px) and (max-width: 1600px) { grid-template-columns: 1fr 1fr 1fr; }
```

Question 2

Comment cibler les écrans de largeur comprise entre 768 et 1 024 pixels en utilisant les media queries ?

```
@media (min-width: 768px) and (max-width: 1024px) { ... }
```

```
@media (width: 768px, 1024px) { ... }
```

```
@media (width: 768px - 1024px) { ... }
```

Question 3

Comment faire en sorte que tous les éléments d'une grille aient la même largeur automatiquement, sans spécifier de largeur explicite pour chaque colonne ?

```
grid-template-columns: auto-fill;
```

```
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
```

```
grid-template-columns: 1fr;
```

Question 4

Comment définir une grille avec 3 colonnes où la deuxième colonne est 2 fois plus large que les 2 autres ?

```
grid-template-columns: 2fr 1fr 1fr;
```

```
grid-template-columns: 1fr 2fr 1fr;
```

```
grid-template-columns: 1fr 1fr 2fr;
```

Question 5

Comment spécifier un espacement entre les éléments d'une grille sans affecter les marges extérieures des éléments ?

```
grid-gap
```

```
gap
```

```
margin
```

Tous droits réservés à STUDI - Reproduction interdite

IV Essentiel

En conclusion, le Grid layout est un outil essentiel pour créer des mises en page flexibles et responsives en développement web. En comprenant les principes de base tels que la grille, les conteneurs et les axes, vous pouvez tirer parti de sa flexibilité et de son contrôle précis sur la disposition des éléments. Les avantages de Grid layout incluent la capacité à créer des mises en page adaptatives à différents appareils et à réorganiser automatiquement les éléments en fonction de la taille de l'écran.

Pour maximiser votre apprentissage, il est important de comprendre la syntaxe de Grid layout et de penser en termes de grille lors de la planification de vos mises en page. Expérimentez différentes configurations de grille et utilisez les outils de débogage des navigateurs pour visualiser et ajuster votre grille en temps réel. N'oubliez pas d'accorder une attention particulière à l'accessibilité en utilisant des balises sémantiques appropriées et en vérifiant la navigation au clavier !

En pratiquant régulièrement avec le Grid layout et en explorant divers scénarios de mise en page, vous développerez une solide compréhension de cette fonctionnalité et serez en mesure de créer des mises en page efficaces et réactives pour vos projets web. N'oubliez pas de consulter la documentation officielle et de rechercher des ressources supplémentaires pour approfondir vos connaissances et rester à jour avec les dernières pratiques de développement web. Avec ces compétences, vous serez bien équipé pour créer des interfaces utilisateur attrayantes et adaptatives.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Vous êtes un développeur et vous devez concevoir un modèle de type blog en utilisant Grid à partir du fichier HTML fourni et en partant des balises sémantiques <header>, <main>, <section>, <article>, <footer>.

Question 1

[solution n°3 p. 21]

Comment pouvez-vous améliorer la mise en page du blog ?

```
<!DOCTYPE html>
<html>

<head>
    <link rel="stylesheet" type="text/css" href="css/style.css">
</head>

<body>
    <header class="header">
        <nav class="navigation">
            <ul>
                <li><a href="#">Accueil</a></li>
                <li><a href="#">Articles</a></li>
                <li><a href="#">À propos</a></li>
                <li><a href="#">Contact</a></li>
            </ul>
        </nav>
        <h1>Mon Blog</h1>
    </header>

    <main class="main-content">
        <section class="article">
            <h2>Titre de l'article</h2>
            <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non finibus augue, et tristique nisi. Sed sagittis risus vitae ipsum fermentum, a tristique ipsum malesuada...</p>
        </section>
        <section class="article">
            <h2>Titre de l'article</h2>
            <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed non finibus augue, et tristique nisi. Sed sagittis risus vitae ipsum fermentum, a tristique ipsum malesuada...</p>
        </section>

        <aside class="sidebar">
            <h3>À propos de l'auteur</h3>
            <p>Informations sur l'auteur...</p>
        </aside>
    </main>

    <footer class="footer">
        <p>&copy; 2023 Mon Blog. Tous droits réservés.</p>
    </footer>
</body>

</html>
```

Question 2

[solution n°4 p. 23]

Comment pouvez-vous améliorer la disposition de la page en jouant avec les media queries ?

Tous droits réservés à STUDI - Reproduction interdite

B. Test

Exercice 1 : Quiz

[solution n°5 p. 23]

Question 1

Comment peut-on adapter une grille CSS aux différents types d'écrans ?

- En utilisant des unités de mesure adaptatives comme « *fr* » et « *minmax()* »
- En utilisant des medias queries
- En utilisant des classes CSS spécifiques

Question 2

Comment peut-on définir une grille avec 3 colonnes pour les écrans larges et 1 seule colonne pour les écrans étroits en CSS Grid ?

- En utilisant la propriété `grid-template-columns: 1fr 1fr 1fr` pour les écrans larges et `grid-template-columns: 1fr` pour les écrans étroits
- En utilisant la propriété `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr))`
- En utilisant la propriété `grid-auto-columns`

Question 3

Quelles unités de mesure sont utiles pour créer des grilles flexibles et adaptables en CSS Grid ?

- **A** Pourcentages
- **B** Pixels
- **C** Fractions (fr)

Question 4

Comment peut-on spécifier l'espacement entre les lignes d'une grille en CSS Grid ?

- En utilisant la propriété `grid-gap`
- En utilisant la propriété `row-gap`
- En utilisant la propriété `margin`

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Comment peut-on centrer des éléments dans une grille CSS ?

En utilisant la propriété `justify-content: center` pour les lignes et `align-items: center` pour les colonnes

En utilisant la propriété `justify-items: center` pour les lignes et `align-content: center` pour les colonnes

En utilisant la propriété `margin: auto` pour les éléments de la grille

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 6]

Question 1

Qu'est-ce que CSS Grid ?

- Une technique pour créer des grilles de données dans Excel
- Un langage de programmation utilisé pour développer des applications web
- ✓ Un module CSS qui permet de créer des mises en page en utilisant des grilles

CSS Grid est un module CSS qui permet de créer des mises en page en utilisant des grilles. Il offre un moyen puissant de structurer et de positionner les éléments sur une page web.

Question 2

Quels sont les 2 composants principaux de CSS Grid ?

- ✓ Lignes et colonnes
- Classes et identifiants
- Divs et spans

Les 2 composants principaux de CSS Grid sont les lignes et les colonnes. Les lignes définissent les horizontales et les colonnes définissent les verticales de la grille.

Question 3

Quelle propriété CSS permet de définir les lignes d'une grille en CSS Grid ?

- ✓ `grid-template-rows`
- `grid-layout`
- `grid-container`

Tous droits réservés à STUDI - Reproduction interdite

La propriété CSS utilisée pour définir une grille en CSS Grid est `grid-template-columns` (ou `grid-template-rows` pour définir les lignes).

Question 4

Quelle est la façon la plus optimisé pour spécifier la taille d'une colonne ou d'une ligne dans CSS Grid ?

En utilisant des pourcentages

En utilisant des pixels

✓ En utilisant des fractions (fr)

Dans CSS Grid, on peut spécifier la taille d'une colonne ou d'une ligne en utilisant des fractions (fr). Par exemple, `1fr` signifie que la colonne ou la ligne occupera une fraction égale de l'espace disponible.

Question 5

Comment peut-on définir un espace entre les éléments d'une grille dans CSS ?

✓ En utilisant la propriété `grid-gap`

En utilisant la propriété `margin`

En utilisant la propriété `padding`

Pour définir un espace entre les éléments d'une grille dans CSS Grid, on utilise la propriété `grid-gap`. Cette propriété permet de spécifier un espacement entre les colonnes et les lignes de la grille.

Solution n°2

[exercice p. 12]

Question 1

Comment définir une grille à 3 colonnes pour les écrans de largeur supérieure à 1 200 pixels en utilisant les media queries ?

✓ `@media (min-width: 1200px) { grid-template-columns: 1fr 1fr 1fr; }`

`@media (max-width: 1200px) { grid-template-columns: 1fr 1fr 1fr; }`

Tous droits réservés à STUDI - Reproduction interdite

```
@media (min-width: 1200px) and (max-width: 1600px) { grid-template-columns: 1fr 1fr 1fr; }
```

La première réponse définit une grille à 3 colonnes pour les écrans de largeur supérieure à 1 200 pixels.

Question 2

Comment cibler les écrans de largeur comprise entre 768 et 1 024 pixels en utilisant les media queries ?

✓ `@media (min-width: 768px) and (max-width: 1024px) { ... }`

`@media (width: 768px, 1024px) { ... }`

`@media (width: 768px - 1024px) { ... }`

`@media (min-width: 768px) and (max-width: 1024px) { ... }` cible les écrans de largeur comprise entre 768 et 1 024 pixels.

Question 3

Comment faire en sorte que tous les éléments d'une grille aient la même largeur automatiquement, sans spécifier de largeur explicite pour chaque colonne ?

`grid-template-columns: auto-fill;`

✓ `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));`

`grid-template-columns: 1fr;`

`grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));` permet aux éléments de la grille d'avoir la même largeur automatiquement tout en respectant une largeur minimale de 200 pixels.

Question 4

Comment définir une grille avec 3 colonnes où la deuxième colonne est 2 fois plus large que les 2 autres ?

`grid-template-columns: 2fr 1fr 1fr;`

✓ `grid-template-columns: 1fr 2fr 1fr;`

Tous droits réservés à STUDI - Reproduction interdite

```
grid-template-columns: 1fr 1fr 2fr;
```

`grid-template-columns: 1fr 2fr 1fr;` définit une grille avec 3 colonnes, où la deuxième colonne est 2 fois plus large que les 2 autres.

Question 5

Comment spécifier un espacement entre les éléments d'une grille sans affecter les marges extérieures des éléments ?

`grid-gap`

✓ `gap`

`margin`

gap est la réponse la plus appropriée pour spécifier un espacement entre les éléments d'une grille sans affecter les marges extérieures des éléments.

[exercice p. 15]

Solution n°3

Voici un code qui va vous permettre d'améliorer la mise en page du blog :

```css
body {
    display: grid;
    grid-template-columns: 1fr;
    grid-template-rows: auto 1fr auto;
    gap: 20px;
    min-height: 100vh;
    margin: 0;
    padding: 20px;
    font-family: Arial, sans-serif;
    background-color: #f9f9f9;
}

header {
    grid-column: 1 / -1;
    padding: 20px;
    background-color: #333;
    color: #fff;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

header h1 {
    margin: 0;
}

nav ul {
    list-style-type: none;
    padding: 0;
```

Tous droits réservés à STUDI - Reproduction interdite

```
    margin: 0;
    display: flex;
    align-items: center;
}

nav ul li {
    margin-right: 10px;
}

.main-content {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    gap: 20px;
}

nav ul li a {
    text-decoration: none;
    color: #ffffff;
}

.article {
    display: grid;
    grid-template-columns: 1fr;
    padding: 20px;
    background-color: #fff;
}

.article h2 {
    margin: 0;
}

.article p {
    margin-bottom: 20px;
}

.sidebar {
    padding: 20px;
    background-color: #2e2424;
    color: #f9f9f9
}

.sidebar h3 {
    margin: 0 0 10px;
}

footer {
    grid-column: 1 / -1;
    padding: 20px;
    background-color: #333;
    color: #fff;
    text-align: center;
}
```


Tous droits réservés à STUDI - Reproduction interdite

[exercice p. 15]

Solution n°4

Voici une réponse qui aurait pu convenir ici :

```
@media (max-width: 768px) {
    .main-content {
        grid-template-columns: 1fr;
    }
    .article {
        grid-template-columns: 1fr;
    }
    .sidebar {
        grid-template-columns: 1fr;
    }
}
```

Solution n°5

[exercice p. 16]

Question 1

Comment peut-on adapter une grille CSS aux différents types d'écrans ?

- En utilisant des unités de mesure adaptatives comme « *fr* » et « *minmax()* »
- ✓ En utilisant des medias queries
- En utilisant des classes CSS spécifiques

Pour adapter une grille CSS aux différents types d'écrans, on peut utiliser des medias queries. Les medias queries permettent d'appliquer des règles spécifiques en fonction des caractéristiques de l'écran, comme sa largeur, sa hauteur ou son orientation.

Question 2

Comment peut-on définir une grille avec 3 colonnes pour les écrans larges et 1 seule colonne pour les écrans étroits en CSS Grid ?

- ✓ En utilisant la propriété `grid-template-columns: 1fr 1fr 1fr` pour les écrans larges et `grid-template-columns: 1fr` pour les écrans étroits
- En utilisant la propriété `grid-template-columns: repeat(auto-fit, minmax(200px, 1fr))`
- En utilisant la propriété `grid-auto-columns`

Pour définir une grille avec 3 colonnes pour les écrans larges et 1 seule colonne pour les écrans étroits en CSS Grid, on peut utiliser des medias queries. Par exemple, on peut définir la propriété `grid-template-columns: 1fr 1fr 1fr` pour les écrans larges et `grid-template-columns: 1fr` pour les écrans étroits.

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Quelles unités de mesure sont utiles pour créer des grilles flexibles et adaptables en CSS Grid ?

- **A** Pourcentages
- **B** Pixels
- **C** Fractions (fr)

Les unités de mesure utiles pour créer des grilles flexibles et adaptables en CSS Grid sont les pourcentages et les fractions (fr). Les pourcentages permettent de définir des proportions relatives, tandis que les fractions (fr) permettent de spécifier une largeur flexible basée sur la taille disponible.

Question 4

Comment peut-on spécifier l'espacement entre les lignes d'une grille en CSS Grid ?

- En utilisant la propriété `grid-gap`
- ✓ En utilisant la propriété `row-gap`
- En utilisant la propriété `margin`

Pour spécifier l'espacement entre les lignes d'une grille en CSS Grid, on utilise la propriété `row-gap`. Cette propriété permet de définir un espacement vertical entre les lignes de la grille.

Question 5

Comment peut-on centrer des éléments dans une grille CSS ?

- En utilisant la propriété `justify-content: center` pour les lignes et `align-items: center` pour les colonnes
- ✓ En utilisant la propriété `justify-items: center` pour les lignes et `align-content: center` pour les colonnes
- En utilisant la propriété `margin: auto` pour les éléments de la grille

Pour centrer des éléments dans une grille CSS Grid, on peut utiliser les propriétés justify-items: center pour les lignes et align-content: center pour les colonnes. Ces propriétés permettent de centrer les éléments horizontalement et verticalement dans leurs cellules respectives de la grille.

Tous droits réservés à STUDI - Reproduction interdite